

1. What is Python, why is it used in AI, and what are its main advantages?

Answer:

Python is a high-level, interpreted programming language. It is widely used in AI, Machine Learning, and Data Science because its syntax is simple and it has powerful libraries such as NumPy, Matplotlib, Pandas, Scikit-learn, and TensorFlow. Python is dynamically typed, so we do not need to declare variable data types manually.

2. Explain variables, assignment, and the main Python data types used in your program.

Answer:

A variable stores data in memory and is created using the assignment operator =.

```
name = "Shifaullah"  
age = 19  
weight = 75.5  
is_student = True
```

The main data types are:

```
int      = Whole numbers  
float    = Decimal numbers  
str      = Text  
bool     = True or False
```

Python automatically detects the data type.

3. Explain the arithmetic operators +, -, *, /, and %.

Answer:

These operators perform mathematical calculations.

```
+ Addition
```

- Subtraction
- * Multiplication
- / Division
- % Modulus or remainder

Example:

```
a = 10
b = 3

print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a % b)
```

10 % 3 gives 1 because the remainder is 1.

4. What is a dataset, and how did you create an AI-style dataset using Lists and Dictionaries?

Answer:

A dataset is a collection of related data used for analysis or AI processing.

Example using Lists:

```
ages = [18, 19, 20, 21]
weights = [60, 70, 80, 90]
results = ["Fit", "Fit", "Overweight", "Overweight"]
```

Example using a Dictionary:

```
dataset = {
    "Age": [18, 19, 20, 21],
    "Weight": [60, 70, 80, 90],
    "Result": ["Fit", "Fit", "Overweight", "Overweight"]
}
```

Age and Weight are features, while Result is the label or output.

5. What is NumPy, how do you import it, and why is it useful in AI?

Answer:

NumPy is a Python library used for numerical calculations and array operations.

```
import numpy as np
```

A NumPy array is created using:

```
marks = np.array([70, 85, 90, 75, 80])
```

NumPy is useful in AI because AI algorithms perform large mathematical, numerical, vector, and matrix calculations.

6. What is Matplotlib, how is it imported, and what functions were used to draw charts?

Answer:

Matplotlib is a Python library used for data visualization.

```
import matplotlib.pyplot as plt
```

Important functions are:

<code>plt.plot()</code>	Line chart
<code>plt.bar()</code>	Bar chart
<code>plt.title()</code>	Chart title
<code>plt.xlabel()</code>	X-axis label
<code>plt.ylabel()</code>	Y-axis label
<code>plt.show()</code>	Display chart

Example:

```
plt.plot(students, marks, marker="o")  
plt.show()
```

7. Explain the difference between a line chart and a bar chart and when each should be used.

Answer:

A line chart is mainly used to show trends or changes across ordered data.

A bar chart is used to compare values between different categories.

For student marks, a bar chart is better for direct comparison, while a line chart visually shows the pattern of marks.

8. What is a List? Explain its main properties and important insertion, deletion, and update operations.

Answer:

A List is an ordered, mutable collection that allows duplicate values.

```
marks = [70, 85, 90, 75]
```

Insertion:

```
marks.append(95)
marks.insert(1, 80)
marks.extend([60, 65])
```

Update:

```
marks[0] = 100
```

Deletion:

```
marks.remove(85)
marks.pop()
del marks[1]
```

Lists are useful when data needs frequent modification.

9. Explain the main List functions such as index, count, sort, reverse, copy, and clear.

Answer:

```
marks.index(90)
```

Finds the index of a value.

```
marks.count(85)
```

Counts occurrences.

```
marks.sort()
```

Sorts ascending.

```
marks.sort(reverse=True)
```

Sorts descending.

```
marks.reverse()
```

Reverses the current order.

```
new_marks = marks.copy()
```

Creates a copy.

```
marks.clear()
```

Removes all values.

We can also use:

```
len()  
max()  
min()  
sum()
```

10. What is indexing, negative indexing, slicing, and membership in Python collections?

Answer:

Indexing accesses a specific value.

```
marks[0]
```

Negative indexing accesses values from the end.

```
marks[-1]
```

Slicing extracts part of a collection.

```
marks[0:3]
```

This takes index 0 to 2.

Membership checks whether a value exists.

```
90 in marks
```

Python uses zero-based indexing.

11. What is a Tuple, how is it different from a List, and how can a Tuple be modified indirectly?

Answer:

A Tuple is an ordered and immutable collection.

```
ages = (18, 19, 20, 21)
```

A List is mutable, but a Tuple cannot be directly changed.

To modify a Tuple:

```
age_list = list(ages)
```

```
age_list.append(22)
ages = tuple(age_list)
```

Tuple functions include:

```
count()
index()
len()
max()
min()
sum()
```

Tuples are useful for fixed data.

12. What is Tuple unpacking and how does it work?

Answer:

Tuple unpacking assigns Tuple values to separate variables.

```
student = ("Ali", 20, 85)

name, age, marks = student
```

Now:

```
name = Ali
age = 20
marks = 85
```

Normally, the number of variables should match the number of Tuple values.

13. What is a Set and what are its main properties and basic operations?

Answer:

A Set is an unordered collection of unique values. It does not allow duplicate elements and does not use normal indexing.

```
class_a = {"Ali", "Ahmed", "Sara"}
```

Add values:

```
class_a.add("Hamza")  
class_a.update(["Umar", "Zain"])
```

Delete values:

```
class_a.remove("Ali")  
class_a.discard("Ahmed")  
class_a.pop()
```

`remove()` gives an error if the value does not exist, while `discard()` does not.

14. Explain Union, Intersection, Difference, and Symmetric Difference in Sets.

Answer:

Union combines all unique values:

```
a.union(b)  
a | b
```

Intersection returns common values:

```
a.intersection(b)  
a & b
```

Difference returns values present in the first Set but not the second:

```
a.difference(b)  
a - b
```

Symmetric Difference returns values present in either Set but not both:

```
a.symmetric_difference(b)  
a ^ b
```

15. Explain `issubset`, `issuperset`, and `isdisjoint`.

Answer:

`issubset()` checks whether all elements of one Set exist in another.

```
a.issubset(b)
```

`issuperset()` checks whether one Set contains all values of another.

```
b.issuperset(a)
```

`isdisjoint()` checks whether two Sets have no common elements.

```
a.isdisjoint(b)
```

16. What is a Dictionary and how do insertion, access, update, and deletion work?

Answer:

A Dictionary stores data in key-value pairs.

```
student = {  
    "name": "Ali",  
    "age": 20,  
    "marks": 85  
}
```

Access:

```
student["name"]  
student.get("age")
```

Insertion:

```
student["city"] = "Abbottabad"
```

Update:

```
student["marks"] = 90  
student.update({"age": 21})
```

Deletion:

```
student.pop("city")  
del student["age"]
```

Dictionary keys must be unique, but values may repeat.

17. Explain the main Dictionary functions `get`, `keys`, `values`, `items`, `popitem`, `setdefault`, and `copy`.

Answer:

```
student.get("age")
```

Retrieves a value safely.

```
student.keys()
```

Returns keys.

```
student.values()
```

Returns values.

```
student.items()
```

Returns key-value pairs.

```
student.popitem()
```

Removes the last inserted pair.

```
student.setdefault("course", "AI")
```

Adds a key with a default value if it does not exist.

```
new_student = student.copy()
```

Creates a copy.

18. Compare List, Tuple, Set, and Dictionary.

Answer:

Data Structure	Main Property
List	Ordered and mutable
Tuple	Ordered and immutable
Set	Unique unordered values

Data Structure	Main Property
Dictionary	Key-value pairs

A List allows duplicates and modification.

A Tuple allows duplicates but is fixed.

A Set removes duplicates and supports Union and Intersection.

A Dictionary stores structured data using keys and values.

19. What is a Graph, and explain nodes, edges, weights, path, start node, and goal node.

Answer:

A Graph is a data structure made of nodes and edges.

```
Node = State or location
Edge = Connection between nodes
Weight = Cost of an edge
Path = Sequence of connected nodes
Start Node = Search starting point
Goal Node = Destination
```

Example:

```
"A": {"B": 4, "C": 2}
```

It means:

```
A to B costs 4
A to C costs 2
```

A weighted graph contains costs on its edges.

20. What is Uniform Cost Search, is it informed or uninformed, and what formula and data structure does it use?

Answer:

Uniform Cost Search, or UCS, is an uninformed search algorithm. It expands the path with the lowest cumulative actual cost.

Its priority is:

$g(n)$

Where $g(n)$ is the actual cost from the start node to the current node.

UCS uses a Priority Queue implemented in Python using:

```
import heapq
```

UCS does not use a heuristic.

21. Explain the UCS priority queue, `heappush`, `heappop`, visited data, and path cost calculation.

Answer:

The UCS Priority Queue stores:

```
(cost, node, path)
```

The initial queue is:

```
[(0, start, [start])]
```

`heapq.heappush()` inserts data into the Priority Queue.

`heapq.heappop()` removes the item with the smallest priority.

Visited nodes and best costs are stored to prevent processing worse paths.

New cost is calculated as:

```
new_cost = cost + edge_cost
```

New path:

```
new_path = path + [neighbor]
```

UCS selects the lowest cumulative path cost, not simply the cheapest immediate edge.

22. What is A* Search, is it informed or uninformed, and explain $g(n)$, $h(n)$, and $f(n)$.

Answer:

A* is an informed search algorithm because it uses heuristic information.

Its formula is:

$$f(n) = g(n) + h(n)$$

Where:

$g(n)$ = Actual cost from start to current node

$h(n)$ = Estimated remaining cost to goal

$f(n)$ = Estimated total cost

A* uses a Priority Queue and usually explores fewer nodes than UCS when a good heuristic is available.

23. What is a heuristic, why is the goal heuristic zero, and explain admissible and consistent heuristics.

Answer:

A heuristic is an estimated cost from the current node to the goal.

Example:

```
heuristic = {  
    "A": 9,  
    "B": 8,
```

```
"C": 7,  
"G": 0  
}
```

The goal heuristic is zero because no remaining cost exists when we are already at the goal.

An admissible heuristic never overestimates the actual cost:

$$h(n) \leq \text{actual remaining cost}$$

A consistent heuristic satisfies:

$$h(n) \leq c(n, n') + h(n')$$

A good heuristic helps A* find an optimal path efficiently.

24. Explain the main difference between UCS, Greedy Best-First Search, and A*.

Answer:

$$\text{UCS} = g(n)$$

$$\text{Greedy Best-First Search} = h(n)$$

$$\text{A}^* = g(n) + h(n)$$

UCS considers only actual path cost.

Greedy considers only estimated remaining cost.

A* combines both actual and heuristic cost.

If all heuristic values are zero:

$$h(n) = 0$$

Then A* behaves like UCS.

25. Compare UCS and A* and explain when they are optimal.

Answer:

UCS	A*
Uninformed	Informed
Uses $g(n)$	Uses $g(n) + h(n)$
No heuristic	Uses heuristic
Priority Queue	Priority Queue
May explore more nodes	Usually explores fewer
Uses actual cost	Uses actual + estimated cost

UCS is optimal with non-negative edge costs.

A* is optimal when an appropriate admissible or consistent heuristic is used.

Both may return the same optimal path, but their visited order can be different.

26. Explain the A* cost calculation using A to B and A to C.

Answer:

For A to B:

$$g(B) = 4$$

$$h(B) = 8$$

$$f(B) = 4 + 8$$

$$f(B) = 12$$

For A to C:

$$g(C) = 2$$

$$h(C) = 7$$

$$f(C) = 2 + 7$$

$$f(C) = 9$$

A* selects C because $9 < 12$.

A* always prioritizes the lowest $f(n)$ value.

27. What is NetworkX, and explain `Graph`, `add_edge`, `positions`, and `edge labels`.

Answer:

NetworkX is a Python library used to create, analyze, and draw Graphs.

```
import networkx as nx
```

Create an undirected Graph:

```
G = nx.Graph()
```

Add a weighted edge:

```
G.add_edge(node, neighbor, weight=cost)
```

`pos` stores drawing coordinates:

```
"A": (0, 2)
```

These coordinates are only for visualization and are not search costs.

Edge weights are displayed using:

```
nx.draw_networkx_edge_labels()
```

28. How is the final optimal path highlighted in the Graph?

Answer:

The final path is converted into consecutive edges using:

```
path_edges = list(zip(path, path[1:]))
```

For:


```
["A", "C", "F", "G"]
```

It creates:

```
(A, C)
(C, F)
(F, G)
```

These path edges are drawn separately with greater width so the optimal path becomes visually clear.

29. What important Python concepts are used inside UCS and A* code?

Answer:

The search programs use:

```
Dictionary = Store Graph and costs
List = Store path and visited order
Tuple = Store Priority Queue values
for loop = Visit neighbours
while loop = Continue search
if statement = Check conditions
Function = Implement algorithm
return = Return result
heapq = Priority Queue
NetworkX = Graph creation
Matplotlib = Graph visualization
```

This means the basic Python data structures are directly used in AI algorithms.

30. Explain the complete working of UCS and A* from dataset to final Graph.

Answer:

First, a weighted Graph dataset is created using a Dictionary. The start and goal nodes are selected. A Priority Queue stores nodes based on cost.

In UCS, nodes are expanded using the lowest actual cumulative cost $g(n)$.

In A*, nodes are expanded using:

$$f(n) = g(n) + h(n)$$

The algorithms visit neighbours, calculate new costs, update better paths, and continue until the goal is found.

The final result contains:

Visited Order
Optimal Path
Total Cost

NetworkX creates the Graph, Matplotlib displays it, and the final optimal path is highlighted.